Phase 2: Planning Phase

Members Names:

1. **Wanga Malinda**
2. **Ronewa Mphephu**
3. **Dembe Mudanabula**
4. **Neliswa Mogashane**
5. **Katekani Nxalati Maluleke**
6. **Nkhensani Nomndeni Baloyi**
7. **Kagiso Mykie Leshika**
8. **George Malesa Karabo Tshabangu**
9. **Duncan Maluleka**
10. **Mochaki Thato Mohlaba**
11. **Ipfi Brandley Khomola**
12. **Reagile Malope**
13. **Otlotleng Morobe**
14. **Mufhatutshedzwa Ramuntshi**
15. **Nsovo Prince Metana**
16. **Reabetswe Molope**

Project Title: TrackServ

Supervisor Name: Nkateko Nkuna

Group Number: 04

Table of Contents

Question 2

2.1 Identification of Need.

The public needs an improved method of reporting municipal service delivery issues that is both interactive and transparent. Currently, the public uses a variety of means to report these issues including etshwane, email, phone calls and manually through various other means. The current reporting methods often do not provide consistent tracking, lack transparency regarding the assignment and/or status of reported issues, and fail to provide timely updates to the public on how the issue was addressed. As a result, many residents feel unsure if they submitted their concern, who it has been assigned to, and whether it has been resolved.

In developing systems for addressing the previously stated needs, another area of need that was determined was sustaining user engagement. Many users submit one report for an issue and then do not follow up on the same issue nor continue to be engaged in the reporting process. The previous statement indicates the need for tools that promote continued user engagement. City employees also require a centralized tool to support operational task sorting of service requests. City employees require a way to receive service request reports beyond just receiving them, but also to assign the issues to specific city employees, prioritize tasks based on urgency and time sensitive requirements, and effectively track progress of each report.

Additionally, city administrators would like analytical tools in order to measure how quickly reported issues were resolved, what areas may have created bottlenecks or delays in resolving reported issues, and to identify any issues that remain open or unresolved over extended periods of time, thus increasing accountability and improving workflow management. Accurate and actionable data are also required to improve the delivery of services. While general geographic locations of issues may be sufficient in many instances, the collection of precise GPS-based location data and visual evidence such as photographs are critical components in assisting municipal teams to properly identify and confirm reported issues, particularly when Page 2 written descriptions alone cannot establish definitive information about either the exact physical location of the reported issue or its level of severity.

2.2 Preliminary Investigation.

During our preliminary investigation, we looked at some of the common problems that people in our community experience with municipal services. We mainly used our own observations of the area and noticed that there are several issues that affect community members on a regular basis. These included water problems, electricity problems, potholes and damaged roads, illegal dumping, sewage problems, and other municipal related issues.

From what we observed, these problems can have a negative effect on people's everyday lives. We also noticed that there is a communication gap between community members and the municipality. People need an easier way to report problems and, at the same time, know what is happening with the issues they have reported. This made us realize that there is a need for a system that can make the reporting process more organized and transparent. Based on these problems, our group developed TrackServ, which is a web-based platform that allows community members to report municipal issues.

When reporting an issue, a user can describe the problem, provide the location where it is happening, upload a picture as proof, and add any additional information that may help explain the issue. After a report has been submitted, municipal staff can receive and review the complaint. The community member can then see that the issue is under review while the municipality is working on it. Once the problem has been fixed, the municipal staff can upload a picture showing that the issue has been attended to. This gives the community evidence that the reported problem has been dealt with.

Overall, our preliminary investigation helped us identify the main areas that TrackServ needed to focus on. These include reporting issues, identifying their locations, tracking the progress of reports, and improving communication between the community and municipal staff. The main goal of TrackServ is to make the reporting process easier and more transparent for both the community and the municipality.

2.3 Feasibility Study

The feasibility study was conducted to determine whether the proposed TrackServ system is practical and achievable from a technical, operational, and economic perspective. It evaluates the availability of the required technologies and skills, how well the system can support its intended users, and whether its expected benefits justify the resources required for development and operation.

2.3.1 Technical Feasibility

TrackServ is technically feasible because the technologies used to develop the system are available and suitable for its requirements. The system is a web-based application developed using React and JavaScript for the front-end interfaces, including the citizen interface and administrative dashboard and staff dashboard. Supabase, using PostgreSQL, is used for database management, including the database schema and Row Level Security (RLS) policies for controlling access to data. The development team has access to the necessary tools, including Visual Studio Code, GitHub, draw.io, and Supabase. These tools support development, version control, database design, and testing. The team also has the technical skills needed to develop and maintain the system. TrackServ uses relational tables and relationships to manage system data. Supabase also provides authentication and security features that support the system's data protection requirements. Since the system can be accessed through computers, tablets, and smartphones using a web browser, users do not require specialised hardware or software. Therefore, TrackServ is considered technically feasible.

2.3.2 Operational Feasibility

TrackServ is operationally feasible because it is designed to meet the needs of its intended users and improve how information and services are managed. The system provides a centralised platform where users can access information and interact with the available features. The system includes user accounts, community posts, scheduling or event-related functionality, the Staff Page, and administrative functions. The Staff Page provides staff members with a dedicated area to access and manage information related to their responsibilities. This can help staff perform their duties while making relevant information easier to manage and retrieve. TrackServ is designed to be simple to use through a familiar web-based interface, meaning users require limited training. Centralising information and activities can also reduce reliance on manual processes and separate communication methods. Therefore, the system can be integrated into the normal activities of its intended users and is considered operationally feasible.

2.3.3 Economic Feasibility

TrackServ is economically feasible because the technologies used for development provide free or low-cost options. Tools such as React, GitHub, Visual Studio Code, draw.io, Canva free tier and Supabase reduce the need for significant initial investment during development and testing. The main potential costs include internet access, hosting or database services, maintenance, and infrastructure such as a domain name if the system is deployed on a larger scale. Since TrackServ is web-based, users can access it using existing computers, tablets, or smartphones, reducing the need for specialised hardware. The expected benefits include improved access to information, better communication, centralised data management, reduced reliance on manual processes, and improved transparency between the municipality and the community. Considering the relatively low initial development costs and the potential benefits, TrackServ is considered economically feasible.

2.3.4 Overall Feasibility Conclusion

Based on the technical, operational, and economic assessment, TrackServ is considered a feasible project. The required technologies and skills are available, the system can support its intended users and staff, and the expected benefits justify the resources required for development and operation. Therefore, TrackServ can be successfully developed, implemented, and maintained.

2.4 Project Planning

The TrackServ project will be planned and managed through an iterative SDLC. Various phases will be put in place to facilitate successful development, testing, and implementation of the software.

In the first phase, planning and requirements gathering will take place and needs of the community and municipality will be determined. In the second phase, design and development of the system and user interface/user experience will take place. This phase will also see the design of the user dashboard and reporting interface.

In the third phase, development of system and programming will occur where frontend, backend, database, and application programming interfaces will be created. The fourth phase will involve the implementation and deployment of the system which will include the integration of geolocation and mapping functions.

After development, the system will go through user testing with 30 community members and 5 local NGOs. The areas of interest during testing will include usability, security, performance, and the reporting capability of the users about municipal problems. Maintenance and review will be done in the last stage of the project to determine problems in the system.

2.6 SOFTWARE REQUIREMENT SPECIFICATION

2.6.1

2.6.1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the TrackServ Civic Reporting Platform. It translates the business need established in the TrackServ Business Analysis Document and the TrackServ Project Proposal into a precise, testable statement of what the software must do, the constraints under which it must operate, and the external interfaces it must support.

The document is intended for the project team, the academic supervisor, municipal stakeholders, field-service staff, and NGO partners. It forms the technical baseline against which the delivered citizen portal, administrative dashboard, and staff application are built, verified, and accepted.

2.6.1.2 Scope

TrackServ is a three-application, web-based civic engagement and municipal service management platform. It consists of:

1. A public citizen portal for reporting and following municipal service issues.
2. A separate administrative dashboard for municipal administrators to triage, assign, analyse, and manage reports.
3. A staff application for municipal field workers to view assigned issues, attend to them, update their progress, upload completion evidence, and record resolution outcomes.

All three applications share a single Supabase PostgreSQL backend. Supabase provides authentication, database access, object storage for media, and real-time change-data-capture subscriptions. Changes made in one application are made available to the other authorised applications through the shared backend and real-time synchronisation.

TrackServ will allow residents to submit environmental and infrastructure issues, such as road damage, water leaks, illegal dumping, and broken public infrastructure. Reports may include photographs, an automatically captured GPS location validated against the municipal boundary, a category, severity, and written description.

The platform will allow administrators to view reports on a live operations map, filter and prioritise them, assign them to staff members or groups, schedule field visits, manage statuses, and monitor performance through analytics. Field staff will be able to log in securely, view their assigned work, start attendance when they arrive at an issue location, update the work status, upload proof of completed work, add resolution notes, and mark the assigned issue as resolved.

The following are explicitly outside the scope of this release:

- Native mobile applications for iOS and Android.
- Integration with existing municipal legacy systems.
- Biometric authentication.
- Multi-municipality configuration; the initial rollout targets Tshwane Municipality.
- Commercial billing, advertising payments, or pricing arrangements as part of the core civic reporting workflow.
- Automated email or SMS notifications requiring a separate server-side messaging service.

2.6.2

2.6.2.1 Product Perspective

TrackServ is a new, self-contained product rather than a component of an existing municipal system. It is delivered as three React web applications served over HTTPS:

- The citizen portal provides public-facing reporting, tracking, community participation, and profile functions.
- The administrative dashboard provides municipal management, triage, assignment, scheduling, and analytics functions.
- The staff application provides a focused field-service workspace for municipal employees who attend to assigned reports.

All three applications read from and write to the same Supabase project. Supabase provides the PostgreSQL database, authentication, object storage for photographs and proof images, and real-time subscriptions. There is no bespoke middle-tier API layer; the applications use the auto-generated Supabase API and database functions, with access control enforced through row-level security policies and application role checks.

The citizen portal, administrative dashboard, and staff application have separate login and navigation experiences. This separation limits each user type to the workflows appropriate to its role and prevents administrative and field-service routes from being exposed as part of the public citizen experience.

The administrative dashboard is intended for supervisors and municipal administrators who need an overview of all reports and operational performance. The staff application is intended for field workers who need a practical view of only the reports assigned to them and the actions required to resolve those reports.

The three applications interact through the following shared workflow:

1. A citizen submits a report through the citizen portal.
2. The report appears in the administrative dashboard through the shared database and real-time subscription.
3. An administrator reviews the report, sets its priority, and assigns it to a staff member or group.
4. The assigned report appears in the staff member's application.
5. The staff member starts attendance, performs the work, uploads evidence, and records a resolution note.
6. The report status is updated to resolved and the result becomes visible to authorised administrators and to the reporting citizen.

2.6.2.2 Product Functions

The TrackServ platform shall provide the following functions.

Citizen portal functions:

- Registration, authentication, logout, password recovery, and profile management for citizens.
- Guided issue reporting with category, severity, description, additional information, photographs, and location.
- GPS capture, map selection, location search, reverse geocoding, and municipal-boundary validation.
- Validation that a report location falls within Tshwane Municipality before submission.
- A personal dashboard showing the current status of every report submitted by the citizen.
- Display of report history, categories, locations, submission dates, statuses, and resolution information.
- Community upvoting of existing reports to indicate public priority.
- A badge or recognition system for continued participation and reporting activity.
- Community posts, likes, saved posts, comments, and profile information.
- Access to the services marketplace where applicable to the release configuration.
- Real-time display of authorised status changes made by administrators or staff.

Administrative dashboard functions:

- Secure administrator authentication and role-based access control.
- Display of all authorised reports on a live operations map.
- Street and satellite map views where supported by the mapping service.
- Search and filtering by title, description, location, category, status, severity, assignee, and date.
- Prioritisation of reports using severity, age, overdue status, and community vote count.
- Manual report entry for issues received through meetings, telephone calls, social media, or other non-platform channels.
- Assignment of reports to individual staff members or staff groups.
- Updating of report status through the municipal workflow.
- Scheduling and management of field events associated with reports.
- Viewing of staff workload and assignment distribution.
- Viewing of report photographs, staff-uploaded proof images, and resolution notes.
- Analytics covering report volume, category distribution, resolution time, resolution rate, overdue reports, and workload by category or staff member.
- Real-time updates when citizens submit reports or staff and other administrators change report data.

Staff application functions:

- Secure staff authentication and role validation.

    - Staff profile display including name, username, role, and profile image where available.
    - Display of only the reports assigned to the authenticated staff member, unless the staff role grants additional access.
    - Display of assigned issue details including report number, title, category, description, location, coordinates, severity, photographs, assignment date, and current status.
    - Map view showing assigned issue locations and the staff member's relevant work area.
    - Sorting and filtering assigned issues by status, severity, category, date, and location.
    - Starting attendance when the staff member arrives at the issue location.
    - Recording the attendance or work-start timestamp in the report workflow.
    - Updating assigned issues to an in-progress state.
    - Viewing scheduled events or visits associated with assigned reports.
    - Uploading completion or proof photographs after work has been performed.
    - Recording a resolution note and attendant name.
    - Updating an issue to resolved after completion evidence and resolution details have been submitted.
    - Viewing a work history of assigned, in-progress, resolved, and completed issues.
    - Displaying the latest authorised report and assignment changes without requiring a manual refresh.
    - Preventing staff members from assigning reports to themselves or other staff unless that permission is explicitly granted by their role.

Shared platform and synchronisation functions:

- Shared storage of reports, profiles, categories, media, assignments, events, and resolution records in Supabase PostgreSQL.
- Supabase Auth authentication for citizens, administrators, and staff.
- Supabase Storage support for report photographs, profile images, community images, and completion evidence.
- Row-level security and role-based access control for citizen, administrator, and staff data access.
- Real-time bidirectional synchronisation between the citizen portal, administrative dashboard, and staff application.
- Propagation of new citizen reports to the administrative dashboard.
- Propagation of new assignments from the administrative dashboard to the assigned staff member's application.
- Propagation of staff attendance, progress, completion evidence, and resolution status to authorised administrators.
- Propagation of final status changes to the reporting citizen's dashboard.
- Audit timestamps for report creation, assignment, attendance, updates, event scheduling, and resolution.

2.6.2.3 User Roles

Citizen:

A citizen can create and manage their own reports, view report progress, vote on reports, participate in the community, and manage their own profile. A citizen cannot access administrative or staff workflows.

Administrator:

An administrator can view and manage all authorised reports, categories, locations, assignments, events, staff workloads, and analytics. Administrators can assign work and update report statuses according to municipal policy.

Staff member:

A staff member can view and work on reports assigned to their profile. A staff member can start attendance, update work progress, upload proof, add resolution notes, and complete assigned issues. Staff access does not automatically grant access to all reports or administrative analytics.

2.6.2.4 External Interfaces

User interface

The platform shall provide responsive interfaces for desktop, tablet, and mobile browser sizes. The citizen, administrator, and staff interfaces shall expose only the navigation and actions relevant to the authenticated user's role.

Mapping interface

The platform shall support interactive maps for citizen location selection, citizen report visualisation, administrator operations monitoring, and staff assigned-issue navigation. Mapping may use OpenStreetMap, Esri, Leaflet, or equivalent supported services.

Authentication interface

The platform shall use Supabase Auth for registration, login, logout, password recovery, session management, and role-aware access checks.

Storage interface

The platform shall use Supabase Storage for report images, profile images, community images, service listing images, and staff completion evidence.

Real-time interface

The platform shall use Supabase PostgreSQL change-data-capture subscriptions to refresh authorised report, assignment, event, and resolution views when data changes.

2.6.2.5 Principal Non-Functional Requirements

Security

- All applications shall be served over HTTPS in deployment environments that support production use.
- Database row-level security shall restrict access according to authenticated user and role.
- Citizens shall access their own private report and profile data only.
- Staff members shall access assigned reports and permitted staff workflow data only.
- Administrators shall access management and analytics data according to their administrative role.
- Uploaded files shall be subject to type, size, ownership, and access controls.

Performance

- Normal report, assignment, and staff-workflow screens should load without unnecessary full-page refreshes.
- Real-time updates should appear within seconds under normal network conditions.
- The system should remain usable on mobile devices and slower connections.
- Image loading, map rendering, and report lists should be optimised before a large production rollout.

Availability

- The citizen portal should always be available for reporting subject to Supabase, hosting, network, and mapping-service availability.
- Temporary failures in image upload or real-time subscriptions shall display a clear error and shall not silently discard report or resolution data.

Usability

- Citizen report submission shall guide users through category, description, location, media, and confirmation steps.
- Administrative screens shall support rapid scanning, filtering, assignment, and prioritisation.
- Staff screens shall prioritise assigned work, location, attendance, completion evidence, and status updates.
- Status, severity, assignment, overdue, and resolution states shall be visually distinguishable.

Scalability

- The database design shall support increasing report, image, user, staff, assignment, and event volumes.
- The platform shall be load-tested with a realistic dataset before wider municipal rollout.
- Realtime subscription usage across all three applications shall be tested with concurrent users.

Maintainability

- The three applications shall use consistent report statuses, category identifiers, role names, and shared Supabase relationships.
- Database migrations and generated database types should be maintained as part of future development.
- Business rules that affect security, validation, voting, assignments, and status changes should be enforced at the database or trusted server boundary and not only in client-side code.

2.6.3 Operating Environment

- Client: current versions of Chrome, Edge, Firefox and Safari on desktop, tablet and mobile devices, with layouts adapting across the 480px, 640px, 768px and 1024px breakpoints.
- Front end: React web applications, served as static assets over HTTPS.
- Back end: Supabase managed PostgreSQL, Supabase Auth, Supabase Storage and Supabase Realtime.
- Mapping and geospatial services: browser Geolocation API, reverse geocoding, GeoJSON municipal boundaries, and OpenStreetMap and Esri tile layers for street and satellite views.
- Network: the platform assumes intermittent but generally available mobile internet access in the target community.

2.6.4 Design and Implementation Constraints

- Access is web-based only in this phase; no native mobile applications are delivered.
- The rollout is limited to a single municipality; boundaries and branding are not yet configurable for reuse elsewhere.
- The platform depends on Supabase as a managed backend, which constrains database, authentication and storage design to that provider's capabilities.
- There is no dedicated API layer, so all authorisation logic must be expressible as row-level security policies together with client-side and server-side validation.
- Personal and geolocation data must be handled in line with POPIA and the published TrackServ Terms and Conditions.
- Delivery is bound by the academic project timeline and the resources of Group 04.

2.6.5 Assumptions and Dependencies

- The municipality continues to act on submitted reports, sustaining the value proposition of the platform.
- NGO partners assist with promotion and community awareness.
- Reliable internet access and GPS-capable devices are available to a sufficient proportion of the target community.
- Hosting, database capacity, storage, security tooling and support are funded and owned as ongoing operational responsibilities.
- Third-party mapping, tile and reverse-geocoding services remain available under acceptable terms.

2.6.6 Specific Requirements

Requirements are uniquely identified. Functional requirements carry the prefix FR, external interface requirements the prefix EI, and non-functional requirements the prefix NFR (Section 4). Priority is stated as Must, Should or Could.

2.6.6.1 Functional Requirements

2.6.6.1.1 User account and authentication

ID	Requirement	Priority	Source
FR-01	The system shall allow a community member to register an account using an email address and password, and shall verify the email address before the account is activated.	Must	BR-01
FR-02	The system shall authenticate citizens and municipal staff through separate login flows, and shall not expose administrative routes within the public application.	Must	BR-03
FR-03	The system shall allow a user to reset a forgotten password through a time-limited link sent to the registered email address.	Must	BR-01
FR-04	The system shall allow a user to view and update their own profile, and to request deletion of their account and associated personal data.	Must	T&Cs
FR-05	The system shall support two-factor authentication for administrative accounts and shall enforce it for all accounts with administrative privileges.	Must	NFR-03
FR-06	The system shall restrict every administrative function to an authenticated account holding a municipal staff role.	Must	BR-03

2.6.6.1.2 Issue reporting

ID	Requirement	Priority	Source
FR-07	The system shall allow an authenticated citizen to submit a report consisting of a title, description, category, severity, location and at least one photograph.	Must	BR-01
FR-08	The system shall capture the reporter’s GPS position automatically at the time of submission and shall allow the reporter to adjust the pinned location on a map before submitting.	Must	Proposal 7.1

ID	Requirement	Priority	Source
FR-09	The system shall reverse-geocode the selected coordinates into a readable street address and display it to the reporter for confirmation.	Must	Proposal 7.1
FR-10	The system shall validate the selected coordinates against the municipal boundary and shall reject, with an explanatory message, any report falling outside that boundary.	Must	Proposal 7.1
FR-11	The system shall accept image uploads in common web formats up to a defined maximum file size, and shall reject files that exceed it or fail validation.	Must	Proposal 10
FR-12	The system shall assign every accepted report a unique reference and an initial status of Open, and shall record the submission timestamp.	Must	BR-02
FR-13	The system shall present the reporter with existing nearby reports of the same category before submission, so that a duplicate can be supported by an upvote instead of being re-reported.	Should	Proposal 7.3

2.6.6.1.3 Tracking and community engagement

ID	Requirement	Priority	Source
FR-14	The system shall provide each citizen with a personal dashboard listing all reports they have submitted, with current status, category, location and last update.	Must	BR-02
FR-15	The system shall update a citizen’s view of a report within seconds of an administrator changing its status, without requiring a page refresh.	Must	Proposal 9
FR-16	The system shall allow an authenticated citizen to upvote an existing report once, and shall display the aggregate number of upvotes.	Should	Proposal 7.3
FR-17	The system shall award badges to citizens for defined participation milestones and display them on the user’s profile.	Could	Proposal 7.3
FR-18	The system shall never display a reporter’s account name, email address or exact residential location to other community members.	Must	BR-06

2.6.6.1.4 Administrative dashboard

ID	Requirement	Priority	Source
FR-19	The system shall display all reports on a live operations map with street and satellite view options and status-coded markers.	Must	Proposal 8.1
FR-20	The system shall allow administrators to filter and search reports by status, category, severity, date range, assigned staff member and location.	Must	BR-04
FR-21	The system shall allow an administrator to open a report and view its full detail, including photographs, coordinates, address and complete status history.	Must	BR-04
FR-22	The system shall allow an administrator to change a report's status through the defined workflow of Open, In Progress, Resolved and Rejected, and shall require a comment when a report is rejected.	Must	BR-04
FR-23	The system shall allow an administrator to assign a report to a staff member and to reassign it, recording who made the change and when.	Must	Proposal 8.4
FR-24	The system shall allow an administrator to create a report manually on behalf of an issue raised through another channel, with status defaulting to Open.	Must	Proposal 8.6
FR-25	The system shall record a resolution outcome and closure note against every report marked Resolved.	Must	BR-04
FR-26	The system shall write an immutable audit entry for every administrative action taken on a report.	Should	BA §12

2.6.6.1.5 Analytics and reporting

ID	Requirement	Priority	Source
FR-27	The system shall present administrators with an analytics dashboard reporting average resolution time, resolution rate and the number and proportion of overdue reports.	Must	BR-05
FR-28	The system shall present workload distribution by category and by assigned staff member.	Should	Proposal 8.5
FR-29	The system shall flag a report as overdue when it has remained unresolved beyond the target resolution time defined for its severity.	Must	BR-05

ID	Requirement	Priority	Source
FR-30	The system shall allow analytics results to be filtered by date range and exported for reporting purposes.	Could	Proposal 8.5

2.6.6.1.6 Staff Application

ID	Requirement	Priority	Source
FR-31	The system shall provide a separate authenticated staff application and shall permit access only to users whose profile has an authorised staff role.	Must	Proposal 6
FR-32	The system shall display each staff member's assigned reports and shall restrict the default staff view to reports assigned to the authenticated staff member.	Must	Proposal 8.4
FR-33	The system shall allow a staff member to open an assigned report and view its title, category, description, location, coordinates, severity, photographs, assignment date, and current status.	Must	Proposal 8.4
FR-34	The system shall display assigned report locations on an interactive map and shall allow the staff member to identify the location of each assigned issue.	Should	Proposal 8.1
FR-35	The system shall allow a staff member to filter and sort assigned reports by status, severity, category, date, and location.	Should	BR-04
FR-36	The system shall allow a staff member to start attendance for an assigned report and shall record the attendance or work-start timestamp.	Must	Proposal 9
FR-37	When attendance is started, the system shall update the assigned report to In Progress and shall make the change visible to authorised administrators in real time.	Must	Proposal 9
FR-38	The system shall allow a staff member to view scheduled visits or field events associated with an assigned report.	Should	Proposal 8.4
FR-39	The system shall allow a staff member to upload one or more completion or proof photographs for an assigned report and shall associate each photograph with that report.	Must	Proposal 7.1
FR-40	The system shall allow a staff member to enter a resolution note and attendant name when completing an assigned report.	Must	Proposal 8.4

ID	Requirement	Priority	Source
FR-41	The system shall allow a staff member to mark an assigned report as Resolved only after the completion workflow has recorded the resolution details and shall make the resolution visible to the reporting citizen and authorised administrators.	Must	Proposal 9
FR-42	The system shall provide a staff work-history view showing assigned, in-progress, resolved, and completed reports, including status and attendance or completion information.	Should	Staff Workflow
FR-43	The system shall notify the staff application in real time when an administrator assigns, reassigns, or updates an authorised report assigned to that staff member.	Must	Proposal 9; SRS 2.6.2.2
FR-44	The system shall prevent a staff member from viewing or modifying reports that are not assigned to them unless their role explicitly grants broader access.	Must	SRS 2.6.2.3; Security requirements
FR-45	The system shall record the staff member, timestamp, status change, uploaded evidence, and resolution details for each staff action performed on an assigned report.	Must	BA §12; Proposal 9

2.6.6.2 External Interface Requirements

2.6.6.2.1 User interfaces

- EI-01: Both applications shall present a responsive layout adapting across the 480px, 640px, 768px and 1024px breakpoints, with maps and complex layouts stacking vertically on mobile.
- EI-02: A shared component library (logo, header, footer, sidebar) shall keep branding and navigation behaviour consistent across all pages.
- EI-03: The navigation sidebar shall collapse into a mobile menu on small viewports.
- EI-04: All interactive controls shall be keyboard reachable and carry accessible labels, and colour shall not be the sole means of conveying report status.

2.6.6.2.2 Hardware interfaces

- EI-05: The citizen portal shall use the device Geolocation API to obtain GPS coordinates, and shall degrade gracefully to manual map selection where the device denies or cannot provide a position.
- EI-06: The citizen portal shall accept photographs from the device camera or local file storage.3.2.3 Software interfaces
- EI-07: Both applications shall interact with the Supabase auto-generated PostgreSQL API for all data operations.
- EI-08: Authentication shall be performed through Supabase Auth, with sessions issued and refreshed by that service.
- EI-09: Report media shall be stored in Supabase Storage in an authenticated, access-controlled bucket.
- EI-10: Real-time updates shall be delivered through Supabase Realtime change-data-capture subscriptions on the reports table.
- EI-11: Map rendering shall use OpenStreetMap and Esri tile services, and address resolution shall use a reverse-geocoding service.
- EI-12: Municipal boundary validation shall be performed against a GeoJSON boundary definition for the target municipality.

2.6.6.2.3 Software interfaces

- EI-07: Both applications shall interact with the Supabase auto-generated PostgreSQL API for all data operations.
- EI-08: Authentication shall be performed through Supabase Auth, with sessions issued and refreshed by that service.
- EI-09: Report media shall be stored in Supabase Storage in an authenticated, access-controlled bucket.
- EI-10: Real-time updates shall be delivered through Supabase Realtime change-data-capture subscriptions on the reports table.

- EI-11: Map rendering shall use OpenStreetMap and Esri tile services, and address resolution shall use a reverse-geocoding service.
- EI-12: Municipal boundary validation shall be performed against a GeoJSON boundary definition for the target municipality.

2.6.6.2.4 Communications interfaces

- EI-13: All client-server communication shall take place over HTTPS with a valid certificate.
- EI-14: Transactional email (verification, password reset and status notification) shall be delivered through the configured email service.

2.6.7 Non-Functional Requirements

2.6.7.1 Performance

- NFR-01: Interactive pages shall render usable content within three seconds on a standard mobile connection.
- NFR-02: A status change made in the administrative dashboard shall be visible on the affected citizen dashboard within five seconds.
- NFR-03: The operations map, report list and analytics module shall remain responsive against a dataset of at least 1,000 reports of varying status and age.
- NFR-04: Uploaded images shall be compressed and resized, and media and non-critical views shall be lazily loaded.

2.6.7.2 Availability and Reliability

- NFR-05: The platform shall remain available 24/7, with a target uptime of 99% per calendar month.
- NFR-06: The database shall be backed up daily, with a documented restore procedure and a defined recovery point objective.
- NFR-07: The platform shall be rebuildable and redeployable from source and backup in the event of an emergency or system failure.
- NFR-08: Loss of a third-party mapping or geocoding service shall not prevent report submission; the system shall fall back to manual coordinate entry.

2.6.7.3 Security

- NFR-09: All personal data shall be transmitted over HTTPS and stored encrypted at rest.
- NFR-10: Two-factor authentication shall be available and enforced for administrative accounts.
- NFR-11: Access control shall be enforced at the database layer through row-level security policies, in addition to application-level checks.
- NFR-12: All input shall be validated server-side as well as client-side, and submissions shall be rate limited to resist abuse and automated flooding.
- NFR-13: Passwords shall be stored only as salted hashes, and session tokens shall expire and be refreshed securely.
- NFR-14: Administrative actions shall be logged with the acting account, action and timestamp.

2.6.7.4 Privacy and Compliance

- NFR-15: Personal and geolocation data shall be processed in line with the Protection of Personal Information Act (POPIA).
- NFR-16: Only the personal data necessary to operate the service shall be collected, and consent shall be captured at registration.

- NFR-17: Reports shall be anonymous to the public while remaining traceable internally for accountability and follow-up.
- NFR-18: Data retention periods, terms of use and account deletion rights shall be published and accessible to all users.

2.6.7.5 Usability and Accessibility

- NFR-19: A first-time user shall be able to submit a complete report without assistance in under five minutes.
- NFR-20: Error messages shall state plainly what went wrong and what the user should do next.
- NFR-21: The interface shall meet WCAG 2.1 Level AA colour contrast and keyboard navigation expectations.
- NFR-22: The platform shall remain usable on low-cost devices and constrained data connections so that vulnerable residents are not excluded.

2.6.7.6 Maintainability and Portability

- NFR-23: Shared branding and layout components shall be maintained in a single reusable library across both applications.
- NFR-24: Automated test coverage shall be provided for the critical paths of both the citizen portal and the administrative dashboard.
- NFR-25: Municipal boundaries, categories, severities and target resolution times shall be held as configuration rather than hard-coded values, to permit reuse by other municipalities.
- NFR-26: The applications shall run on any static web host and shall not depend on a specific operating system on the client.

2.6.8 Security, Legal and Ethical Considerations

TrackServ processes personal data, photographs and precise location data, so security and privacy are treated as requirements rather than as implementation details. The following controls are mandatory for acceptance:

- Row-level security policies restrict every table so that a citizen can read and write only their own records, while municipal staff roles receive the wider access their duties require.
- Server-side validation and rate limiting protect report submission and voting from automated abuse and malicious flooding.
- Photographs and location data are stored in an authenticated, access-controlled store and are never published with identifying user details.
- Anonymity to the public is preserved for all reports, regardless of how the report was submitted.
- Terms of use, data retention periods and account deletion rights are published and accessible to all users.
- Accessibility and affordability of access are considered so that the platform does not exclude vulnerable residents.

2.6.8.1 Risks Affecting the Requirements Baseline

Risk	Description	Requirement-level mitigation
Low adoption	Residents continue to prefer traditional in-person reporting.	Usability and accessibility requirements (NFR-19 to NFR-22) and engagement features FR-16 and FR-17.
False or malicious reports	The platform is used to submit inaccurate or abusive reports.	Authenticated submission (FR-07), duplicate prompting (FR-13), rejection with comment (FR-22) and rate limiting (NFR-12).
Account compromise	An administrative account is taken over.	Enforced two-factor authentication (FR-05, NFR-10) and administrative action logging (FR-26, NFR-14).
Regulatory non-compliance	Mishandling of personal or geolocation data breaches POPIA.	Data minimisation, encryption, consent capture and published retention (NFR-15 to NFR-18).
Operational continuity	Hosting, storage and maintenance are not resourced, interrupting service.	Availability, backup and rebuild requirements (NFR-05 to NFR-07).
Performance degradation	Growing report volumes slow the map, dashboard and analytics.	Performance requirements NFR-01 to NFR-04 and load testing under Section 7.

2.7 Data Models

2.7.1 TrackServ consists of three user-facing applications connected to one Supabase PostgreSQL backend:

1. Citizen application

Citizens register, report municipal issues, track progress, vote on reports, participate in the community, manage profiles, and browse the services marketplace.

2. Administrative application

Municipal administrators manage reports, locations, categories, assignments, staff workloads, scheduled events, and analytics.

3. Staff application

Municipal field staff log in separately, view issues assigned to them, start attendance when they arrive, update work status, upload completion evidence, record resolution notes, and review their work history.

All primary keys are expected to use UUID values. Timestamps are expected to use PostgreSQL timestamptz values stored in UTC. Supabase Auth owns the authentication identity records.

2.7.2. IDENTITY AND ACCESS MODELS

2.7.2.1 auth.users

This is the Supabase-managed authentication table. It stores login identities and authentication information.

Fields:

id

UUID primary key. Identifies the authenticated user.

email

Text email address used for login.

user_metadata

JSON metadata. Registration code reads the optional full name from this metadata.

authentication timestamps

Supabase-managed values such as account creation, confirmation, and last sign-in times.

Relationships:

One auth.users record has one profiles record.

An auth.users record may own reports, posts, listings, votes, likes, saves, comments, reviews, and payments.

2.7.2.2 profile

This is the application profile associated with a Supabase Auth user. It is used by citizens, administrators, and field staff.

Fields

id

UUID primary key. Also references auth.users.id.

full_name

Text display name

username

Text account or public username.

email

Text application-level email copy.

phone

Text optional contact number.

location

Text stated user location. New profiles use Tshwane Municipality as the client-side default.

about

Text optional biography.

profile_picture

Text URL for the user's avatar.

role

Text access role. The applications use citizen and staff roles. The administrator login also expects an administrator role.

email_notifications

Boolean indicating whether email notifications are enabled.

public_profile

Boolean indicating whether the profile can be publicly viewed.

created_at

Timestamptz profile creation/member-since date.

updated_at

Timestamptz last profile update, if configured.

Relationships:

A profile can create many reports.

A profile can be assigned many reports when the profile has the staff role.

A profile can create many community posts, comments, likes, saved posts, listings, reviews, and payments.

A profile can schedule many events.

2.7.3. CIVIC REPORTING MODELS

2.7.3.1 categories

Reference data used by both the citizen and administrative applications.

Fields:

id

UUID primary key.

category_name

Text category label. Examples include Roads & Infrastructure, Water Leak, Electricity, Garbage, and Other.

description

Text optional category description.

created_at

Timestamptz creation time, if configured.

2.7.3.2 reports

The reports table is the central model in TrackServ. Reports can be submitted by citizens or entered manually by administrators for issues received through meetings, social media, or other channels.

Fields:

id

UUID primary key.

user_id

UUID creator or reporter. References profiles.id and the authenticated user.

category_id

UUID reference to categories.id.

title

Text report title. Citizen submissions may not provide a title; the interface can derive one from the description and category.

description

Text main issue description. The citizen interface limits this to 500 characters.

additional_information

Text optional extra information. The citizen interface limits this to 300 characters.

location

Text readable address or location description.

latitude

Numeric latitude captured from GPS, search, or map selection.

longitude

Numeric longitude captured from GPS, search, or map selection.

status

Text workflow state. Observed values are open, in_progress, under_review, resolved, closed, and rejected.

severity

Text priority or severity. Observed values are Low, Medium, and High.

votes

Integer denormalised count of community votes.

assigned_to

UUID assigned staff profile. References profiles.id.

assigned_to_group_id

UUID optional assigned team or group. References groups.id when enabled.

assigned_a

Timestamptz time of assignment.

started_at

Timestamptz time field staff started attendance or work.

proof_image_url

Text URL for completion or proof evidence uploaded by administrators or staff.

resolution_notes

Text completion notes entered by an administrator.

created_at

Timestamptz report submission or creation time.

updated_at

Timestamptz last status, assignment, or report update.

Report workflow:

open -> under_review -> in_progress / reject-> resolved -> closed

rejected is a terminal exception. The applications can return an issue to open. The administrative application uses severity for assignment priority and may highlight the highest-voted active report as high priority.

2.7.3.3 report_images

Stores images associated with reports. Citizen reports support up to five images, with a documented maximum of 5 MB per image. Staff can upload completion evidence using the same table.

Fields:

id

UUID primary key.

report_id

UUID reference to reports.id.

image_url

Text Supabase Storage public URL.

uploaded_at

Timestamptz upload time.

2.7.3.4 report_votes

Join model that records a user's vote on a report.

Fields

id

UUID primary key.

report_id

UUID reference to reports.id.

user_id

UUID reference to profiles.id and auth.users.id.

created_at

Timestamptz vote time.

The database enforces one vote per user per report with a unique constraint on report_id and user_id. The citizen application calls toggle_report_vote to add or remove a vote and update reports.votes.

2.7.3.5 issue_resolutions

Detailed field-work and completion record used by the staff application.

Fields:

id

UUID primary key.

report_id

UUID reference to reports.id. The staff application uses one current resolution record for a report.

attendant_name

Text name of the staff member who attended to the issue.

resolution_note

Text completion note entered by staff.

resolution_image_url

Text URL of the resolution or completion image.

attended_at

Timestamptz time attendance or completion was recorded.

created_at

Timestamptz record creation time, if configured.

updated_at

Timestamptz last update time, if configured.

2.7.4. STAFF APPLICATION MODELS AND WORKFLOW

2.7.4.1 Staff account

The staff application uses Supabase Auth for login. After authentication, it loads the staff member's profile from profiles.

Required staff account data:

profiles.id

Authenticated staff user ID.

profiles.full_name

Staff display name shown in the header, issue details, and resolution records.

profiles.username

Staff username used as a fallback display name.

profiles.role

Must be staff for the staff application and for report assignment selection.

The administrative application lists profiles where role equals staff and stores the selected profile ID in reports.assigned_to.

2.7.4.2 Staff assigned issue view

The staff application loads reports where reports.assigned_to equals the authenticated staff profile ID.

The staff issue view uses:

reports.id

Issue identifier

reports.title

Issue title.

reports.description

Issue description.

reports.location

Location where the staff member must work.

reports.latitude and reports.longitude

Coordinates used for the operations map.

reports.status

Current work state.

reports.severity

Issue priority.

reports.created_at

Date the issue was reported.

reports.assigned_at

Date the issue was assigned.

reports.proof_image_url

Completion evidence stored directly on the report when the admin workflow uses that field.

categories.category_name

Issue category.

report_images.image_url

Report and completion images.

The staff application displays assigned issues as cards, an issue detail panel, an interactive map, and a work history table.

2.7.4.3 Staff attendance workflow

When staff start work on an issue, the staff application updates reports with:

status = in_progress

started_at = current timestamp

If started_at is not supported by the database, the application falls back to updating only the status.

2.7.4.4 Staff completion workflow

When staff finish an issue, the application performs these operations:

1. Upload the completion image to the images Storage bucket.

2. Insert a report_images record containing the report ID and image URL.

3. Create or update the issue_resolutions record.

4. Store attendant_name, resolution_note, resolution_image_url, and attended_at.

5. Update reports.status to resolved.

The completion workflow therefore keeps the report summary, image evidence, and detailed resolution record connected.

2.7.4.5 Staff work history

The work history is derived from reports assigned to the staff member. It displays the issue number, category, location, check-in time, check-out or completion time, and current status.

Check-in is represented by issue_resolutions.attended_at or reports.started_at, depending on the view. The repository does not show a separate check-in/check-out table.

2.7.4.6 Staff and administrator responsibilities

Administrator:

- Views all reports.

- Filters reports by category and status.

- Assigns reports to staff profiles.

- Changes report status.

- Schedules field events.

- Reviews workload and analytics.

Staff member:

- Views only reports assigned to their profile.

- Starts attendance when arriving at the location.

- Updates work progress.

- Uploads completion evidence.

- Records resolution notes.

- Marks work as resolved through the completion workflow.

2.7.5. COMMUNITY MODELS

2.7.5.1 community_posts

Fields:

id

UUID primary key.

user_id

UUID author reference to profiles.id.

category_id

UUID optional reference to categories.id.

title

Text post title.

content

Text post body.

image_url

Text optional image URL stored in the images bucket.

post_type

Text post format, such as text, photo/video, poll, event, or report update.

visibility

Text audience visibility setting.

status

Text publication state. The community feed currently loads published posts.

created_at

Timestamptz creation or publication time.

updated_at

Timestamptz last edit time, if configured.

2.7.5.2 likes

Join model for post likes.

Fields:

id: UUID primary key.

post_id: UUID reference to community_posts.id.

user_id: UUID reference to profiles.id.

created_at: Timestamptz like time.

2.7.5.3 saved_posts

Join model for saved posts.

Fields:

id: UUID primary key.

post_id: UUID reference to community_posts.id.

user_id: UUID reference to profiles.id.

created_at: Timestamptz save time.

2.7.5.4 comments

Fields:

id: UUID primary key.

post_id: UUID reference to community_posts.id.

user_id: UUID author reference to profiles.id.

content: Text comment body.

created_at: Timestamptz comment time.

updated_at: Timestamptz last edit time, if configured.

2.7.6. ADMINISTRATION AND SCHEDULED WORK MODELS

2.7.6.1 Assignments

Assignments are implemented on reports using

assigned_to

Staff profile ID.

assigned_to_group_id

Optional team/group ID.

assigned_at

Assignment timestamp.

status

Current assignment/work state.

severity

Assignment priority displayed as Low, Medium, or High.

2.7.6.2 groups

The admin report query references groups.id and groups.name through the reports_assigned_to_group_id_fkey relationship.

fields:

id: UUID primary key.

name: Text group name.

2.7.6.3 events

Events represent scheduled field visits or other work associated with a report.

Fields:

id: UUID primary key.

report_id: UUID reference to reports.id.

event_name: Text event or visit name.

description: Text event description.

location: Text visit location.

event_date: Date or timestamptz scheduled date.

start_date: Timestamptz event start.

end_date: Timestamptz event end.

status: Text event state.

notes: Text staff notes.

created_by: UUID profile or authenticated user that scheduled the event.

created_at: Timestamptz creation time.

updated_at: Timestamptz last update time.

2.7.7. STORAGE MODELS

Supabase Storage stores files. Database tables store URLs or paths instead of binary file contents.

2.7.7.1 images bucket

Used for report images, staff completion evidence, community post images, and service listing images.

Report path format:

<report_id>/<index>-<timestamp>.<extension>

The administrative application may also use a proof filename containing the report ID and timestamp.

2.7.7.2 Profile bucket

Used for profile avatars.

Observed path format:

<user_id>/avatar.<extension>

The profile application uploads avatar files with upsert enabled.

2.7.8. RELATIONSHIP SUMMARY

auth.users has one profiles record.

profiles has many reports as reporter.

profiles has many reports as assigned staff member.

profiles has many community_posts.

profiles has many comments, likes, saved_posts, service_listings, listing_reviews.

profiles has many events as event creator.

categories has many reports.

categories has many community_posts.

reports has many report_images.

reports has many report_votes.

reports has one or more issue_resolutions over its lifecycle.

reports has many events.

groups has many reports when group assignment is enabled.

2.7.9. DERIVED APPLICATION MODELS

These values are calculated by the applications and may not be stored as columns:

- Overdue report: An active/open report older than seven days.
- Top-voted report: Active report with the highest votes count.
- Resolution time: Time calculated from report creation or work start until resolution.
- Resolution rate: Resolved and closed reports divided by reports in the selected analysis period.
- Overdue rate: Overdue reports divided by reports in the selected analysis period.
- Category breakdown: grouped by category.
- Team workload: Reports grouped by assigned staff, assigned group, status, and category.

2.7.10. VALIDATION AND SECURITY REQUIREMENTS

The database and server-side functions enforce the following requirements:

- Uses row-level security for citizen-owned records and staff/admin operations.

- Allows report assignment only to staff profiles.

- Enforces one report vote per user per report.

- Enforces one like and one saved-post record per user per post.

- Validates report descriptions, categories, status, severity, coordinates, and image limits server-side.

- Validates report coordinates against the Tshwane Municipality GeoJSON boundary.

- Restricts media access according to ownership and staff permissions.

- Records actor identities and timestamps for status, assignment, event, and resolution changes.

- Ensures staff users can read and update only the reports assigned to them unless their role grants broader access.

2.5

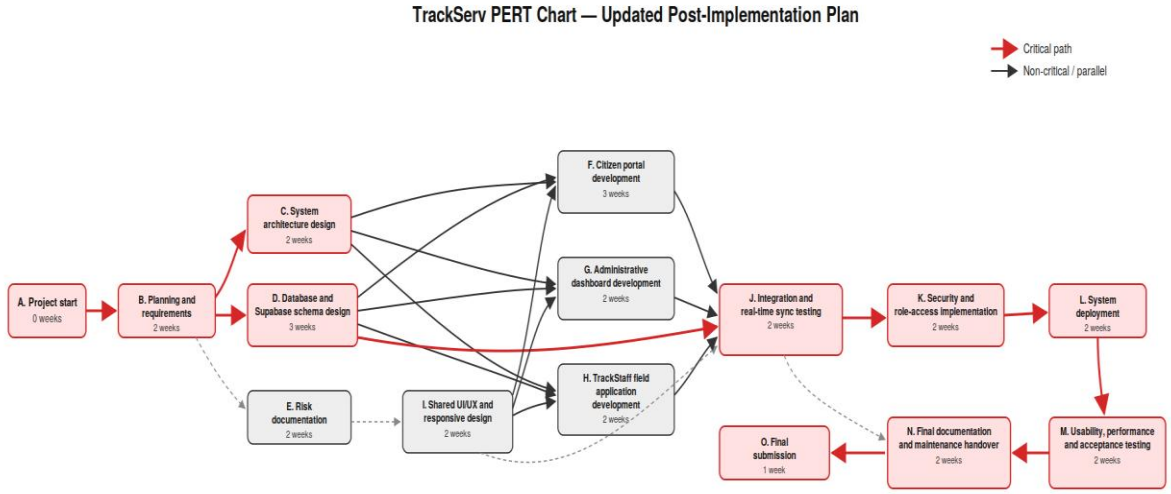
2.5.1 Gantt Chart

Task	APR				MA	
	w1	w2	w3	w4	w1	w2
PHASE 1 - PLANNING & REQUIREMENTS						
Team Setup						
Requirement Gathering						
Project Proposal						
Project Charter						
Risk documentation						
Milestone Approved				☑		
PHASE 2 - SYSTEM DESIGN						
System Architecture Design						
Database Schema Design						
UI/UX wirefram						
Interactive Prototype						
Milestone Approved						
PHASE 3 - DEVELOPMENT						
FrontEnd Dev (React.js)						
Backend Dev (Supabase / PostgresSQL)						
Database Implematention (SQL)						
Security Implementation (RLS / Policies)						
Integration Testing						
PHASE 4 - IMPLEMENTATION & DEV						
System Deployment						
test and fix bugs						
PHASE 5 — USER ACCEPTANCE TESTING						
UAT with 30 Users						
Feedback Analysis & Fixes						
Milestone Approved						
PHASE 6 - DOCUMENTATION						
Final Documentation						
Final Report & Submission						
SUBMISSION						

AY		JUN				JUL		
w3	w4	w1	w2	w3	w4	w1	w2	w3
		☑						

	AUG				SEP			
w4	w1	w2	w3	w4	w1	w2	w3	w4
✓								
				✓				
					✓			
								✓

2.5.2 Pert Chart



Critical path:

A → B → C/D → J → K → L → M → N → O

(C and D run in parallel; both must finish before J starts)

Parallel delivery streams (must all complete before J):

F. Citizen portal development, G. Administrative dashboard development,
H. TrackStaff field application development — enabled by I. Shared UI/UX (from E. Risk documentation)

Dashed grey arrows indicate sequencing/informational dependencies that are not on the timed critical path.